

COOPER | CODEFEST AD ASTRA 2026 — Informe técnico · Etapa 1

Base de entrega: 98d391b | Auditoría: 12 de agosto de 2026 Evidencia calculada desde los artefactos auditados de la entrega.

Nota de versión: este documento es la transcripción íntegra de [informe_tecnico.pdf](#), con las ampliaciones de redacción solicitadas integradas en su sección correspondiente. Cada ampliación queda marcada explícitamente como tal. No se reemplaza el PDF original; ambos coexisten como artefactos de la entrega.

211,485	1,813	768	50
chunks indexados	documentos cubiertos	dimensiones	consultas validadas

1. Resumen ejecutivo

Cooper implementa recuperación híbrida multilingüe y no generativa sobre el corpus oficial. La ruta entregada combina un encoder público de Hugging Face, embeddings L2, búsqueda exacta FAISS por producto interno, metadata JSONL alineada y evidencia de un grafo GraphML trazable. No intervienen LLM, decoders, BM25, ChromaDB, query expansion, cross-encoders ni reranking generativo.

El artefacto final contiene 211,485 vectores y cubre 1,813 de 1,837 archivos pertenecientes a los tres fenómenos. Las 50 respuestas cumplen el esquema oficial: tres documentos distintos y diez fragmentos por consulta, cada texto con un máximo de 250 palabras.

Matriz de cumplimiento obligatorio

Requisito	Implementación verificada	Estado
Encoder público y multilingüe	intfloat/multilingual-e5-base ; ES/EN/PT; mismo modelo para pasajes y consultas	CUMPLE
FAISS y similitud coseno	IndexFlatIP exacto; vectores y queries L2; 211,485 filas alineadas	CUMPLE
Complejidad lingüística	Cortes en oraciones; listas, tablas y filas como unidades estructurales	CUMPLE*
Metadata obligatoria	doc_id , chunk_id , fuente, formato, fenómeno, posición, num_tokens y texto	CUMPLE
Salida oficial	50 líneas q001–q050; 3 documentos; 10 fragmentos; máximo 250 palabras	CUMPLE
Grafo bonus	GraphML: 11,653 nodos, 17,003 aristas y evidencia doc/chunk	INCLUIDO
Persistencia y reproducción	faiss.write_index/read_index , JSONL, generador autocontenido y Git LFS	CUMPLE

Conclusión de auditoría. La entrega obligatoria es cargable y válida. El asterisco remite a 369 unidades indivisibles mayores a 480 tokens, registradas de forma trazable; no se cortaron ni se inventaron fronteras.

2. Corpus, extracción y chunking

El inventario registra 1,839 archivos: 1,837 pertenecen a F1/F2/F3 y dos son archivos de control. No hay archivos de cero bytes. Un `doc_id` estable se obtiene del SHA-256 de la ruta relativa, lo que mantiene la identidad entre copias del corpus sin depender de rutas absolutas.

Formato	Archivos corpus	Docs cubiertos	Chunks	Tratamiento
PDF	760	759	70,746	PyMuPDF; OCR solo sin capa textual; elimina boilerplate repetido
JSON	964	946	3,264	Campos <code>title/body/text/...</code> ; listas de párrafos conservan orden y <code>json_path</code>
CSV	26	26	125,956	Una fila semántica con pares <code>columna: valor</code>
XLSX	6	5	1,141	Filas por hoja, cabecera como contexto
PBF	73	73	10,364	Atributos por feature y deduplicación dentro del tile
TXT	1	1	9	Texto UTF-8 normalizado
Imagen	9	3	5	OCR selectivo en tres figuras analíticas; seis decorativas excluidas

Cobertura observada

98.69%	24	5,302	478
1,813 / 1,837 archivos de fenómenos	documentos sin chunks	chunks >250 palabras indexados correctamente	máximo de tokens observado

Los 24 archivos no cubiertos son 18 JSON administrativos sin cuerpo analítico y seis imágenes decorativas (cinco JPG y un AVIF). Los 48 PDF escaneados y tres figuras con texto relevante fueron recuperados mediante OCR trazable; no se indexaron catálogos o fotografías para inflar artificialmente la cobertura. Frente al índice anterior: +51 documentos y duplicados exactos por documento reducidos de 748 a 356.

Política de fragmentación

El objetivo es 360 tokens y el máximo 480. La segmentación respeta párrafos y puntuación terminal; los ítems de lista retienen sus líneas continuadas. Las filas tabulares permanecen completas salvo que una fila exceda el máximo, caso en que se agrupan campos completos separados por punto y coma. Una oración indivisible mayor que 480 tokens se registra y se omite: nunca se corta por posición de token.

Métrica	Tokens/chunk	Palabras/chunk
Mínimo	1	1
Mediana	303	119
P95	358	240
Máximo	478	403
Promedio	298.32	135.77

Aclaración del límite de 250 palabras. No limita el índice. Existen 5,302 chunks válidos con más de 250 palabras y hasta 478 tokens. El límite se aplica solamente a cada fragmento de `resultados.jsonl`, tal como exige la Sección 9.2 del reglamento.

Ampliación — Preprocesamiento de tablas y JSON estructurales

Fragmentación estructurada de CSV y XLSX. Cada fila de un archivo CSV o XLSX se trata como una unidad semántica completa, nunca como texto plano concatenado. La conversión sigue el patrón `columna: valor` por celda, con las cabeceras de columna retenidas como contexto obligatorio de cada fragmento:

```
columna_1: valor_1; columna_2: valor_2; columna_3: valor_3
```

Para XLSX, la fragmentación opera por hoja: cada hoja conserva su propia cabecera como contexto local, evitando que columnas de hojas distintas con nombres iguales se mezclen semánticamente. Una fila se mantiene íntegra salvo que exceda el máximo de 480 tokens, caso en el que se agrupan campos completos separados por punto y coma hasta ajustar el límite, sin cortar un valor de celda a la mitad.

Extracción de JSON por `json_path`. La extracción sobre JSON navega la estructura mediante `json_path`, preservando el origen jerárquico de cada fragmento (campos `title`, `body`, `text` o listas de párrafos, conservando su orden original). Los descriptores administrativos —campos de metadatos que no aportan contenido analítico— se separan del cuerpo del documento y se enrutan directamente a `metadata.jsonl`, sin mezclarse con los chunks de contenido indexado. Esta separación es la razón por la que 18 de los 24 documentos no cubiertos son archivos JSON puramente administrativos sin cuerpo analítico.

Ampliación — Análisis de limitaciones y cumplimiento de restricciones obligatorias

La Sección 3.3 de la especificación técnica prohíbe explícitamente segmentar una oración a través de la frontera de dos fragmentos consecutivos: la completitud lingüística es una restricción obligatoria, no una preferencia de calidad.

De las unidades candidatas totales (211,485 indexadas + 369 omitidas = 211,854), 369 oraciones o unidades indivisibles superaron el máximo de 480 tokens antes de aplicar cualquier corte. Para estos 369 casos, cualquier subdivisión mecánica dentro del límite de 480 tokens habría requerido cortar la unidad por posición de token, violando directamente el requisito de completitud lingüística de la Sección 3.3.

Ante el conflicto entre dos restricciones obligatorias —tope de 480 tokens y prohibición de fragmentación intraoracional— la implementación prioriza la completitud lingüística. Las 369 unidades se registran de forma

trazable y se excluyen del índice final. La exclusión es una decisión de ingeniería deliberada, no una omisión accidental: ninguna oración fue cortada ni se inventaron fronteras artificiales para forzar su inclusión.

Restricción	Origen	Resolución aplicada
Máximo 480 tokens por chunk	Especificación 3.3	Respetada en 211,485/211,854 unidades candidatas
Prohibición de fragmentación intraoracional	Especificación 3.3	Respetada en el 100% de los chunks indexados
Conflicto entre ambas restricciones (369 casos)	Especificación 3.3	Resuelto por exclusión trazable, no por corte mecánico

3. Embeddings, índice y recuperación

Selección del encoder

`intfloat/multilingual-e5-base` ofrece un espacio común para español, inglés y portugués, longitud de entrada compatible con el tope de 480 tokens y una dimensión de 768 que cabe holgadamente en 8 GB de VRAM. Se anteponen los prefijos E5 exactos `passage:` y `query:`. El mismo encoder se usa en indexación y consulta; no existe un decoder en la ruta de recuperación.

Parámetro	Valor final	Implicación
Hardware del build	NVIDIA GeForce RTX 3060 Ti	Embeddings acelerados por CUDA
Precisión	float16	FP16 solo para inferencia CUDA; salida normalizada float32
Batch solicitado / efectivo	16 / 16	Sin reducción por OOM en el build final
Tiempo registrado	15.57 s embeddings; 443.40 s rebuild cacheado	OCR inicial completo: 2,358.9 s; pasadas posteriores usan caché
Caché	1,831 hits de extracción; embeddings=True	Rebuild incremental sin releer/recalcular contenido intacto
OCR	51 documentos; 1,355 chunks; mediana 89.91	Tesseract 5 ES/EN/PT; confianza ≥ 60 ; páginas trazables
Índice	<code>IndexFlatIP</code> ; d=768	Búsqueda exacta; IP equivale a coseno con L2
Grafo	11,653 nodos; 17,003 aristas	Fusión numérica no generativa; evidencia trazable
Tamaños	619.59 MiB FAISS; 326.89 MiB metadata	Artefactos versionados mediante Git LFS

Ampliación — Justificación formal del encoder frente al pliego técnico

La selección de `intfloat/multilingual-e5-base` responde a cuatro criterios exigidos por la especificación, verificados de forma independiente:

Criterio	Exigencia del pliego	Cumplimiento verificado	Estado
Soporte multilingüe	Espacio vectorial único para ES/EN/PT, sin traducción intermedia	Espacio vectorial unificado entrenado sobre corpus multilingüe; prefijos E5 exactos <code>passage:/query:</code> compartidos por las tres lenguas del corpus	CUMPLE
Longitud máxima y dimensiones	Compatibilidad con la ventana de chunking y con el hardware disponible	Tope nativo de 512 tokens del encoder por encima del máximo de 480 tokens de chunking, sin riesgo de truncamiento; 768 dimensiones por vector, footprint compatible con los 8 GB VRAM de la RTX 3060 Ti	CUMPLE
Licencia	Preferencia del pliego por MIT, Apache 2.0 o CC BY	Licencia MIT	CUMPLE
Robustez en recuperación densa	Evidencia de desempeño en benchmarks estandarizados	Modelo evaluado públicamente en las suites MTEB y BEIR sobre tareas de recuperación densa multilingüe	CUMPLE

La sincronización entre el tope de entrada del encoder (512 tokens) y el máximo de chunking (480 tokens) evita truncamiento silencioso: ningún fragmento entregado excede la capacidad de contexto del modelo. La dimensión de 768 por vector mantiene los 211,485 embeddings y el índice `IndexFlatIP` dentro de la VRAM de 8 GB de la RTX 3060 Ti sin reducción de batch. La licencia MIT satisface la restricción de reutilización abierta del pliego sin las condiciones de atribución de CC BY ni la cobertura de patentes de Apache 2.0. El desempeño del encoder en MTEB y BEIR, referencias estándar de la industria para recuperación densa, respalda su robustez semántica cross-lingual sin fine-tuning adicional sobre el corpus del reto.

Flujo de recuperación

`Consulta` → `E5 query` + `L2` → `FAISS top-1000` → `Evidencia GraphML` → `Top-10 / top-3`

FAISS devuelve hasta 1,000 candidatos exactos. Las entidades de la consulta activan aristas del grafo y sus chunks de evidencia reciben un aporte acotado de 0.0025 por evidencia (máximo 0.025); los candidatos exclusivos del grafo se incorporan al pool. Para documentos, las puntuaciones fusionadas se agrupan por `doc_id` mediante max pooling y se seleccionan tres ids distintos. Para fragmentos, se recorre el ranking y se divide la presentación solo en límites lingüísticos hasta completar diez textos de ≤ 250 palabras. Si un chunk origina varios subfragmentos, estos conservan el mismo `chunk_id`, práctica expresamente permitida por la especificación.

Resultados y trazabilidad

Comprobación	Resultado
Consultas y orden	50 líneas; q001 ... q050

Comprobación	Resultado
Documentos	150 objetos; tres <code>doc_id</code> distintos por consulta
Fragmentos	500 objetos; diez por consulta; máximo observado = 250 palabras
Referencias	500/500 <code>chunk_id</code> y <code>doc_id</code> existen en metadata
Subfragmentación	27 repeticiones de <code>chunk_id</code> dentro de consultas; permitidas por reglamento
Texto original	500/500 coincidencias literales tras normalizar espacios; 0 variaciones

Integridad vectorial. `index.ntotal` = metadata = 211,485; no hay `chunk_id` duplicados, todas las secuencias de posición empiezan en 0 y cinco vectores reconstruidos en puntos distribuidos del índice presentan norma L2 = 1.000000.

4. Reproducibilidad, validación y riesgos

Evidencia ejecutada sobre main actualizado

Validación	Resultado	Alcance
Git LFS	OK	Objetos de <code>index.faiss</code> , <code>metadata.jsonl</code> y <code>encoder_config.json</code> íntegros
Compilación	OK	<code>src</code> , <code>scripts</code> , <code>generador.py</code> y tests
Pytest	22 passed	CUDA, L2, FAISS-metadata, OCR/caché, JSON/listas, esquema y grafo
Preflight	PASSED	Carga FAISS, metadata, 50 queries, 3 documentos, 10 fragmentos, ≤250 palabras
Smoke CUDA	OK	Python 3.11.9; torch 2.9.1+cu128; RTX 3060 Ti; E5 + FAISS mínimo
Reproducción	Idéntica	50/50 rankings de documentos; 50/50 rankings de fragmentos idénticos

Reproducción verificada. La regeneración desde la carpeta empaquetada en RTX 3060 Ti y FP16 produjo 50/50 rankings documentales y 50/50 rankings de fragmentos idénticos. La identidad se garantiza para el entorno auditado; otro hardware o dtype puede alterar empates numéricos.

Ampliación — Manual de portabilidad y guía para el jurado

Comando de invocación exacto:

```
python generador.py --consultas consultas.jsonl --base-vectorial ./base_vectorial
--salida resultados.jsonl
```

La identidad de rankings —50/50 documentos y 50/50 fragmentos— está garantizada para el entorno auditado: RTX 3060 Ti, precisión float16 en inferencia CUDA y versiones fijadas de Python 3.11.9 y torch 2.9.1+cu128. Sobre arquitecturas GPU heterogéneas, la resolución de empates numéricos infinitesimales puede variar por redondeo de hardware en la acumulación de punto flotante.

Escenario	Precisión	Garantía
Hardware auditado (RTX 3060 Ti)	float16 inferencia / float32 salida	50/50 rankings idénticos
Hardware heterogéneo	float16	Rankings equivalentes; posibles empates infinitesimales por redondeo
Auditoría numérica estricta inter-GPU	float32 (recomendado)	Elimina la variabilidad de redondeo entre kernels CUDA distintos

Para una auditoría numérica inter-GPU estricta, se recomienda ejecutar la inferencia en precisión simple (float32) en lugar de float16.

Riesgos y limitaciones transparentes

Prioridad	Hallazgo	Impacto / tratamiento
Media	369 oraciones/unidades >480 tokens	Se registran y omiten para cumplir completitud lingüística; revisar manualmente solo si existe una frontera estructural verificable.
Baja	24 archivos sin chunks	18 JSON administrativos sin cuerpo, cinco fotografías JPG y un retrato AVIF. Exclusión deliberada para evitar ruido; no son PDF analíticos perdidos.
Baja	No existe ground truth público	El peso del grafo se eligió con proxies de similitud, diversidad y trazabilidad; validar Recall@k cuando la organización publique juicios de relevancia.
Baja	Empates numéricos entre hardware	La ejecución auditada es idéntica; para auditoría inter-GPU estricta usar FP32 y fijar versiones.
Baja	Entrega separada de main	Los artefactos finales, incluido GraphML, viven en la rama <code>entrega_final</code> . Identificar la revisión final expresamente al entregar o integrarla después de cerrar la competencia.

Huellas SHA-256 para auditoría

Archivo	SHA-256
<code>index.faiss</code>	<code>cc36d8aa375869ae3fffc82be18453aa9b6ea2736990eda90958985f818301e4</code>
<code>metadata.jsonl</code>	<code>6d6da2b162b81d2fc01e17e1cbe83ade2516677bd3f4d9ab2785866105516c2e</code>
<code>resultados.jsonl</code>	<code>28dfacc62955c94bf1f43ed7bdf098de32c05b7f18e0d9ea0abd71b8ef852218</code>
<code>grafo.graphml</code>	<code>8bde8e70a9b72dd59790ee5e0e2a10424489a4a0430fdd3069d8954a101343cb</code>

5. Arquitectura final, bonus y operación

Separación de responsabilidades

Componente	Responsabilidad	Garantía principal
<code>extract.py</code> / <code>chunking.py</code>	Extracción multiformato, OCR selectivo y unidades completas	Texto/página/confianza trazables; máximo 480 tokens
<code>cache.py</code> / <code>vector.py</code>	Caché SHA-256/SQLite, E5, OOM fallback, L2 y FAISS	Mismo espacio semántico y rebuild incremental
<code>retrieval.py</code> / <code>generador.py</code>	Carga, búsqueda, agregación y formato oficial	Top-10 chunks, top-3 documentos, sin generación
<code>validate.py</code> / <code>validate_delivery.py</code>	Esquema, ids, alineación y preflight	Falla temprana ante una entrega incoherente
Git LFS	Distribución de índice y metadata grandes	Clon reproducible sin reindexar

Grafo de conocimiento opcional

La entrega incluye un bonus determinista basado en NetworkX: detecta entidades multilingües por léxico/patrones, extrae relaciones solo cuando existe un verbo disparador y conserva `doc_id/chunk_id` como evidencia. El GraphML versionado contiene 11,653 nodos y 17,003 aristas (6.07 MiB). La auditoría encontró 0 referencias de chunk huérfanas, 0 de documento y 0 aristas sin trazabilidad.

Estado del bonus. `grafo.graphml`, `graph.py` y `retrieval.py` están incluidos; `generador.py` autodetecta el grafo junto al índice. Frente al baseline vectorial coinciden 43/50 rankings documentales y 25/50 rankings de fragmentos: la fusión fue aplicada.

Ampliación — Detalle metodológico y formalización matemática del grafo (bonus)

El módulo bonus se formaliza como un grafo $G = (E, R, T)$, donde E es el conjunto de entidades multilingües detectadas por léxico y patrones (11,653 nodos), $R \subseteq E \times E$ es el conjunto de relaciones explícitas extraídas solo cuando existe un verbo disparador de contexto entre dos entidades (17,003 aristas), y $T : R \rightarrow (doc_id, chunk_id)$ es la función de trazabilidad que ancla cada arista a su evidencia textual de origen. El grafo se serializa en formato GraphML (`grafo.graphml`), preservando E , R y T como atributos de nodo y arista.

La recuperación híbrida no introduce LLM ni decoders en la ruta de inferencia, en cumplimiento estricto de la regla no generativa de la Sección 8.3 de la especificación. El procedimiento es puramente numérico:

1. Las entidades detectadas en la consulta activan sus nodos correspondientes en E .
2. Se consultan los vecinos de primer orden de esas entidades sobre R , obteniendo el conjunto de chunks de evidencia vía T .
3. Cada chunk candidato recibe un incremento acotado de 0.0025 por cada evidencia semántica coincidente, hasta un tope de 0.025 por chunk.
4. El incremento se suma al puntaje de similitud coseno obtenido de FAISS antes del reordenamiento final.

Parámetro	Valor	Restricción satisfecha
Nodos ($ E $)	11,653	—

Parámetro	Valor	Restricción satisfecha
Aristas ($ R $)	17,003	—
Peso por evidencia	0.0025	Fusión numérica, no generativa (Sección 8.3)
Tope de aporte por chunk	0.025 (10 evidencias)	Acota la influencia del grafo frente al ranking vectorial base
Trazabilidad	<code>doc_id</code> + <code>chunk_id</code> por arista	Auditabilidad total de la evidencia

Reproducción operativa en Windows

1. Crear `.venv` con Python 3.11 e instalar torch 2.9.1 desde cu128; luego `requirements.txt`.
2. Ejecutar `setup_ocr.ps1` y los smoke tests de CUDA/OCR.
3. Construir con `build_baseline.py --device cuda --enable-ocr` y el allowlist auditado, o cargar el índice LFS.
4. Generar con el mismo encoder y `--candidates 1000`.
5. Ejecutar pytest, compileall y `validate_delivery.py` hasta obtener `PRECHECK PASSED`.

Decisión final

La entrega satisface las restricciones técnicas centrales de CODEFEST: recuperación vectorial multilingüe con fusión opcional de grafo, sin modelos generativos; índice exacto y persistente, metadata alineada, fragmentos lingüísticamente completos y salida oficial válida. Las mejoras más valiosas frente a la primera iteración son la extracción estructurada de JSON/listas/tablas, el OCR trazable de 48 PDF y tres figuras, la recuperación de unidades largas sin imponer 250 palabras al índice, las cachés por contenido y LFS. Las limitaciones restantes están cuantificadas y no invalidan el preflight; deben guiar la siguiente ronda de calidad, especialmente juicios de relevancia y determinismo FP32 entre hardware.

Resultado. `PRECHECK PASSED` | 22 tests passed | CUDA OK | 211,485 vectores alineados | 50 consultas conformes.